

I fix friction between graphics and code with tools I build myself.

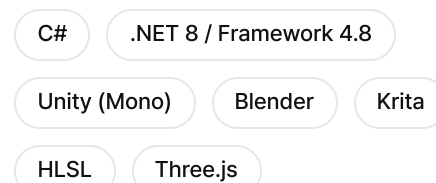
I build games and interactive content with Unity and C#. When texture, shader or 3D work runs into the same snag twice, I build a tool to fix it — that habit has pulled me deeper into graphics, and from there, into reverse engineering.

About

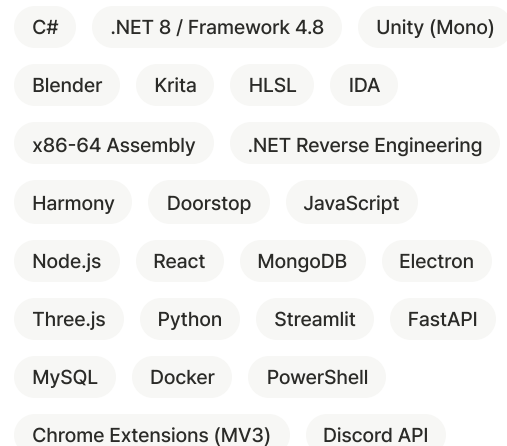
I've always been more curious about how something is made than about the finished result. That curiosity started with game modding.

Solving repeated friction in texture work with tools of my own grew into technical art. Digging into how games and programs are built grew into reverse engineering.

CORE SKILLS



ADDITIONAL TOOLS



Technical Art

I paint skin textures, build a 3D editor for making skins, and pose characters in Blender. Hyperphantasia's model viewer uses neutral lighting, like an FBX viewer, so texture colors read without distortion.

Human: Fall Flat is a game by No Brakes Games and has nothing to do with this site. Game assets, mod executables and source code are not included.

Hyperphantasia

Personal project

v0.4.1 in development

3D skin editor for Human: Fall Flat

PROBLEM

Editing an in-game preset's texture in an external tool like Paint or Krita threw a red or white question-mark error, which made skin work awkward and often slowed things down.

APPROACH

I built a dedicated 3D skin editor in Electron that loads the real FBX model with the game's own UV in Three.js, so a texture edit shows up on the model right away.

TECHNICAL FOCUS

- Real-time 3D viewer built with Three.js inside an Electron app
- Loads the real FBX model with the game's own UV mapping
- RGB masks protect the areas you choose from being painted over
- Neutral, FBX-viewer-level lighting keeps texture colors from reading wrong
- Exports modified parts as PNGs named exactly like the game's own assets, such as ModelFull.png

RESULT

The result is a workflow where I can check how a texture looks on the real model before it ever goes into the game.

Electron

Three.js

FBX

Windows

Next: exploring a locally trained image model (LoRA) on my own skins, with results confined to the mask.

Private repository

Reverse Engineering & Security Tools

Personal research and development tools built to understand how game engines and .NET applications work under the hood. While studying how Unity Mono and .NET load code, I built

a loader, an obfuscator and a trace-based reverse engineering tool.

Human: Fall Flat is a game by No Brakes Games and has nothing to do with this site. Game assets, mod executables and source code are not included.

PLUTO

Personal project

Prototype

A framework for loading code into Unity Mono games

A loading path I built myself in four parts: a core module, a Doorstop entry point, a payload injector and a single-.exe installer.

- At launch, a Doorstop entry point loads code into the Unity engine
- Dependencies such as Harmony are embedded, so one installer file is enough

C#

.NET Framework 4.8

Harmony

Doorstop

Private repository

Dyslexia

Personal project

v1.0.0

Unity-safe .NET rename obfuscator

A conservative obfuscator for Unity Mono assemblies. It writes a dry-run plan first, applies only to a separate output DLL, then verifies the result.

- The input DLL is never overwritten
- Unity callbacks, serialized fields and reflection links are protected, not renamed
- Protection passes such as string encryption are opt-in

C#

.NET 8

Unity Mono

MIT

Private repository

LUNESYS

Personal project

Loading chain and installer

A toolchain of a loading chain where a WinHTTP proxy DLL auto-starts a launcher, an injector and a Discord OAuth installer.

- Keeping the loading chain and the install process apart keeps installing simple
- PowerShell build and deploy scripts

C#

Doorstop

PowerShell

Discord OAuth

Private repository

Project Retrograde

Personal project

MVP in progress

Trace-based reverse engineering tool

Combines static analysis with execution traces to explain why a value or branch came to be. It traces where a value came from and finds where two runs first diverge.

- Loads Windows x86-64 PE files with disassembly, CFG and cross-references
- Keeps observed facts, inferences and confirmed knowledge separate

Python

x86-64

PE

Private repository

Hypercalculia

Personal project

Low-level and reverse-engineering Discord bot

Machine code ↔ assembly, base and IEEE754 conversion, hashing and JWT decoding, available as slash and prefix commands.

Node.js

discord.js

Docker

Private repository

Web, Bots & Automation

A browser extension, a desktop app and Discord bots I made to cut down on repetitive work and admin overhead.

StackFlip

Personal project

v1.2.0

A modular Chromium extension that isolates failure

A Manifest V3 extension for Starblast.io. Each feature is its own module, so when a game update breaks one, the rest keep working.

- A settings panel and a copyable diagnostic report that leaves out sensitive data
- Page-context scripts are separated from the extension context

JavaScript

Chrome MV3

Private repository

Apophenia

Personal project

Coupang × short-form workflow manager bot

A Discord manager bot for a video-production and Coupang-item workflow. It builds the server layout itself, and feature modules load just by adding a file.

Python

discord.py

Slash commands

Private repository

HFF Lobby Logger

Personal project

Multiplayer lobby open/close logger

Polls the public lobby list through the Steamworks API and logs new lobbies as opened and vanished ones as closed to a Discord channel.

Node.js

Steamworks

Discord webhook

Private repository

ShortForm Manager

Personal project

Desktop app for batch short-form uploads

An Electron app that uploads local videos to YouTube, Instagram Reels, TikTok and Facebook at once, using each platform's official API and OAuth.

- OAuth tokens are stored with OS-level encryption
- Both installer and portable .exe builds

Electron

Node.js

OAuth

Private repository

Collaboration Experience

Experience contributing assets and code to a project other people also work in.

Level Construct Studio (LCS)

Collaboration

Asset management for the SinFall scene in a team Unity project

In Level Construct Studio (LCS), a shared Unity project, I handled overall asset management for the SinFall scene. Other scenes were handled by other teammates.

My part	Overall asset management for the SinFall scene
---------	--

Unity

Git

Prefabs

.meta

Private repository

University Coursework

Team and individual projects in AI, web and computer graphics from my major courses. For team projects, I note the part I was responsible for.

Allbirds Clone Coding

University project

MERN-stack e-commerce clone

A responsive shop cloned from the Allbirds website for learning. React, Express and MongoDB cover everything from browsing to ordering and admin tools.

Course	Term project
Period	Nov – Dec 2025
Team	3 people (1 backend, 2 frontend)
My part	Frontend (core logic, UI/UX)

- Filters and sorting, cart, orders and reviews
- Admin page for product registration, discount policy and sales overview

React Vite Express MongoDB

Context API

Brand assets belong to Allbirds, Inc. Built for educational purposes only.

[GitHub ↗](#) Public repository under a teammate's account

GlobalGo

University project

AI-powered overseas job guide

A web app that connects country statistics, job postings and AI Q&A in one flow for people preparing to work abroad.

Course	Creative project
Period	Apr – Jun 2026
Team	5 people
My part	Streamlit UI, data loading, RAG chat integration, original job-posting view

- Country economic indicators kept in MySQL and visualized with Plotly
- An Adaptive RAG pipeline on LangChain, GPT-4o and ChromaDB

Python Streamlit FastAPI MySQL

ChromaDB LangChain GPT-4o

Captured on the repository's CSV data, without the database or API keys. Job search and AI answers need keys, so they don't appear.

Private repository

TechmoUP

University project

Unity 2D climbing game (Computer Graphics)

A playable 2D climbing game built in Unity 6 (URP) and HLSL. I implemented the sprite and buffer-based outline shaders, drag aiming, a boost gauge, and the lighting effects myself.

Course	Computer Graphics
Period	May 2026
Team	Solo (no team)
My part	Sole developer — planning through implementation

- Sprite outline and buffer-based outline shaders (HLSL)
- Drag aiming with a boost gauge, plus sunlight, light rays and flag effects

Unity 6

C#

URP

HLSL

The images and music in the game are AI-generated, and the font is the MapleStory typeface (NEXON). Screens were captured by playing each scene in the Unity editor.

Private repository

Contact

GitHub / @Ultinight ↗

Email / ultinight@naver.com

Discord / ultinight

KakaoTalk / ultinight